A COMPREHENSIVE STUDY REPORT ON

Zero-Shot Text Classification: Foundations, Methods and Current Developments

Natural Language Processing

Assignment 1
Topic 3: Zero-Shot Classification

Chapter 2: Application Topics
Literature reviewed: October 13, 2026

Abstract

Zero-shot text classification assigns passages to categories described at prediction time without training on examples of the target application labels. This report explains the field through its learning settings, natural language inference formulation, contextual representations and major alternative approaches. A BART-MNLI news classifier provides a concrete implementation of passage-hypothesis comparison. The report retains the problem statement, design, proposed methodology, implementation and result structure while connecting each stage to the underlying concepts. It explains candidate-dependent score normalization, independent multiple-topic scoring, input constraints and evaluation measures. Real inference on twelve hand-authored examples produced twelve correct first-ranked classifications; this small demonstration is distinguished from an independent benchmark. The current-status review includes comparative work on embeddings, rerankers and instruction-following models, with representative literature reviewed on October 13, 2026. The discussion identifies label ambiguity, calibration, domain change and reproducibility as continuing challenges. The resulting document presents both an accessible implementation and a framework for understanding why zero-shot flexibility does not remove the need for careful task definition and empirical evaluation.

Keywords: zero-shot classification; NLI; BART; label descriptions; evaluation

Index

Abstract 2

1. Introduction 4

2. Problem Statement 7

3. Design 9

4. Proposed Methodology 11

5. Implementation 14

6. Result 17

7. Current Status and Future Directions 19

8. Reference 21

1. Introduction

Text classification is the process of assigning a passage to one or more categories. It supports activities such as organizing news, identifying customer requests, filtering documents and studying opinions. A passage may be classified by topic, sentiment, emotion or intent. These are different classification aspects: a report about a football defeat may concern sports, express disappointment and describe an event at the same time. A meaningful system must first specify which aspect its categories represent.

Conventional supervised classification learns from labelled examples. A developer collects texts, associates each with an approved category and trains a model to reproduce those assignments. This approach can work well when the label set is stable and representative examples are available. It becomes harder when a new category appears, the domain changes or annotation requires specialist knowledge. Zero-shot classification addresses this situation by using the meaning of category descriptions at prediction time.

1.1 Learning settings and the meaning of zero-shot

In this report, zero-shot means that the application does not train on examples of its target categories before classifying them. The pretrained model still contains knowledge acquired through earlier training. Therefore, zero-shot does not imply an untrained system or prove that every evaluation passage was absent from its pretraining data. The claim concerns the target-task procedure, not complete historical ignorance of the subject.

Few-shot prompting supplies a small number of labelled demonstrations in the model's input. Fine-tuning instead changes model weights using training examples. These operations should be distinguished: a few-shot prompt can leave weights unchanged, while even a small fine-tuning dataset introduces target-task training. Human-written label descriptions are allowed in the zero-shot approach used here, but selecting those descriptions using final test answers would compromise the evaluation.

Table 1. Distinguishing learning settings used in text classification.

Setting	Information provided for the target task	What changes?
Supervised fine-tuning	Labelled training examples	Model weights
Few-shot prompting	A few examples inside the prompt	Input context; weights can remain fixed
Zero-shot application	Task description and candidate labels	Candidate set and input; no target-label training

1.2 Natural language inference

Natural language inference (NLI) compares a premise with a hypothesis. Entailment means the premise supports the hypothesis; contradiction indicates incompatibility; neutrality indicates that support or contradiction is not established. Consider the premise "A student is reading a book." The hypothesis "Someone is reading" is supported, "Nobody is reading" conflicts with it, and "The student is wearing a blue shirt" cannot be settled from the premise alone.

Yin, Hay and Roth presented an entailment formulation for zero-shot text classification across different aspects [1]. In the present application, the passage becomes the premise and each topic becomes a hypothesis. The template "This text is about {}." turns sports into "This text is about sports." A model can then compare the passage with hypotheses for sports, technology, politics and education. These are topic descriptions rather than four newly trained output classes.

The conversion from classification to NLI is useful because the same comparison mechanism can accept different labels. However, topic relevance is not always a strict logical implication. A mixed passage can support several hypotheses, and a vague label can lead to an unclear comparison. NLI scoring is a modelling strategy that still requires application-level evaluation.

1.3 Model selection and application

BART uses a Transformer encoder-decoder architecture and was developed through denoising pretraining [2]. The selected facebook/bart-large-mnli checkpoint was subsequently adapted to MultiNLI [3, 4]. This prior NLI training supplies the comparison capability. Our application uses the saved weights for inference and contains no training loop, optimizer or weight-update step.

The initial application is a short English news classifier. Its four topics provide an understandable demonstration, but the field is broader than news. An intent classifier might compare requests with labels such as password reset and delivery enquiry. An emotion classifier might use joy and anger. Changing the classification aspect requires coherent descriptions and a new evaluation; it is not evidence that the old demonstration result transfers automatically.

1.4 Tokens, representations and attention

A tokenizer converts text into numeric token identifiers. Tokens may be complete words, fragments or punctuation. The model represents them as vectors and combines contextual information through its Transformer layers. Attention allows information at one position to influence another. This helps distinguish words used differently in different sentences, rather than treating every occurrence as an identical keyword.

For this application, the essential input is a passage-hypothesis pair. The tokenizer adds the model's required special markers and prepares the pair for inference. Both pieces consume the finite input window. A long label description can therefore make a passage too long even when the passage alone would fit. The project checks the complete encoded pair rather than counting characters.

1.5 Development of the field and applications

Rule-based categorization assigns categories through manually written conditions. Statistical classifiers commonly learn from word or document features. Pretrained language models add contextual representations and transfer from earlier learning. Zero-shot methods extend this transfer by making labels meaningful text inputs rather than only fixed numeric identifiers. This progression changes the relationship between a developer, a label set and the model; it does not make older approaches universally unsuitable.

Potential applications include organizing an evolving document collection, routing support requests and exploring an initial category system before annotation. The method is especially useful when obtaining labels is difficult, but it should be compared with simpler alternatives when a stable task and good training data exist. A flexible label interface is an advantage only if the resulting decisions are sufficiently reliable for the intended use.

2. Problem Statement

The problem is to categorize a short English passage using a user-supplied set of topic descriptions without training a dedicated classifier on those application categories. The system must validate the inputs, compare candidates consistently and return a ranking that preserves each category's association with its score. The educational objective is to understand how natural-language label descriptions can turn a general NLI model into an application-specific classifier.

2.1 Objectives

The objectives are to explain the target-task zero-shot setting, design a reusable NLI-based classifier, implement input and length validation, support both single-topic and independent topic scoring, and interpret the resulting scores. A further objective is to distinguish a working demonstration from a reliable empirical evaluation. Recorded predictions should identify the checkpoint and configuration so that another reader can examine what was actually run.

2.2 Scope and constraints

The implemented system accepts non-empty English text and at least two unique non-empty labels. It removes surrounding whitespace and rejects labels that differ only in capitalization. It initially uses sports, technology, politics and education, but accepts other candidate topics. Labels should describe comparable concepts: combining sports with positive mixes topic and sentiment and makes the ranking difficult to interpret.

The backend supports multi_label=False and multi_label=True. The former makes topics compete; the latter scores each independently. The interface displays scores in multiple-topic mode and does not implement an automatic selection threshold. Long-document aggregation, multilingual validation and automatic rejection of unsuitable candidate sets are outside the implemented scope. They are discussed as possible extensions rather than existing features.

2.3 Functional and practical requirements

Table 2. Requirements of the implemented application.

Requirement	Specification
Input	A passage and at least two distinct topic descriptions
Validation	Reject blank inputs, repeated labels and overlength pairs
Processing	Compare the passage with one hypothesis per label
Output	Ranked labels, scores, model version and original passage
Execution	Local pretrained-model inference on CPU by default
Presentation	Browser interface, command line and JSON export

The first prediction may require a model download and initialization. Later calls reuse the model. A download failure is an execution issue, not a negative classification result. Memory use, input length and candidate count should be treated as design constraints. Removing task-specific training does not remove the cost of inference or the need to define categories carefully.

3. Design

3.1 System architecture

The application separates presentation, validation, inference and result recording. The browser page and command-line tool both call the same NewsClassifier backend. This separation keeps the classification method consistent and makes it possible to inspect the backend without first learning the interface framework.

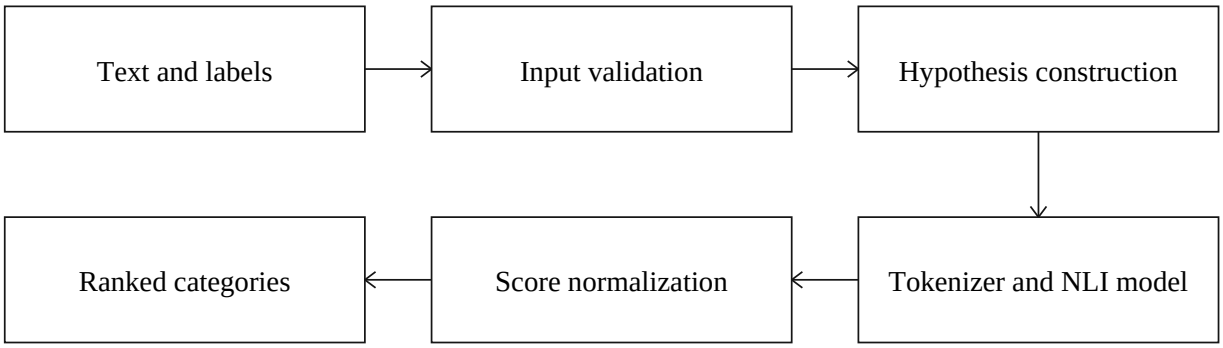


Figure 1. Processing architecture for NLI-based zero-shot classification.

The input layer collects a passage and candidate descriptions. Validation cleans the data and rejects invalid requests. The inference layer loads or reuses the checkpoint, checks each encoded pair and calls the Transformers pipeline. The output layer displays aligned label-score pairs and saves the configuration with the predictions. This structure also permits tests of input handling without repeatedly loading a large model.

3.2 Representation and output

For passage x and candidate c_i, the hypothesis h_i is obtained by inserting c_i into a fixed template. The backend returns a dictionary containing sequence, labels, scores, model, revision, hypothesis_template, multi_label and inference_seconds. Ranked labels and their scores must be interpreted together. Sorting one list independently would attach scores to the wrong categories.

The returned revision identifies the saved checkpoint, while the template records the linguistic formulation of the task. The duration measures the pipeline call after model loading and input-length checks. It is not total first-run latency. A result file therefore supports inspection of both predictions and the conditions under which they were obtained.

3.3 Design rationale

The model is loaded lazily: the object starts without a pipeline and loads it when a valid request needs inference. It is then reused. Streamlit caches the backend object across interface reruns. Reuse reduces repeated initialization, although total memory consumption still depends on the checkpoint and runtime. CPU execution makes the educational workflow usable without assuming access to a CUDA GPU.

The application preserves the sentence's word order, punctuation and negation. Stemming and stop-word removal are not applied because they can change meaning in a passage-hypothesis comparison. For example, removing not can reverse the interpretation of a sentence. The implementation delegates tokenization and model execution to a library while retaining explicit application checks.

3.4 Alternative approaches and trade-offs

An NLI cross-encoder processes the passage and label description together. An embedding method encodes each into a vector and compares their similarity; stored label vectors can be reused. A reranker scores text-description pairs using a learned relevance function. An instruction-following language model can instead be prompted to return a category. Recent comparative research evaluates these families together [7].

Table 3. Major approaches discussed in the field; only NLI is implemented here.

Approach	Main mechanism	Practical consideration
NLI cross-encoder	Joint passage-hypothesis inference	One comparison per candidate; wording matters
Embedding comparison	Similarity between text and label vectors	Label vectors can be reused; similarity is not entailment
Reranker	Learned relevance score for each pair	Needs a checkpoint suited to the intended comparison
Instruction prompting	Generate a label from a task description	Output must be constrained and checked

There is no universal winner independent of task, resources and evaluation. A system handling thousands of categories may shortlist labels with embeddings and rerank a smaller set. That is an architectural extension, not a feature of the current four-topic application. Any shortlist introduces the risk of removing the correct category before the final comparison.

4. Proposed Methodology

4.1 Data preparation and hypothesis construction

The procedure starts with an English passage and a coherent candidate set. Whitespace is cleaned, but the text's semantic structure is preserved. Every candidate is inserted into the same template. Consistency matters: using a detailed description for one topic and an unrelated grammatical form for another may introduce differences beyond their intended category meanings.

For the passage "The team won the final cricket match," the sports hypothesis describes its subject matter. Technology and politics are weaker reference topics in this illustrative example. An unclear passage should not be edited to make the model appear successful. Instead, uncertainty in the human reference label should be documented before interpreting the prediction.

4.2 Inference and single-label scoring

The model produces NLI logits, which are raw numerical outputs. For at least two candidates in single-label mode, the pipeline uses a softmax over their entailment logits [5]. Let e_i denote the entailment logit for candidate i and K the number of candidates. The normalized score and selected category are:

$$s_i = exp(e_i) / \Sigma_{j=1}^{K} exp(e_j) \quad (1)$$

$$predicted\ category = candidate\ with\ the\ largest\ s_i \quad (2)$$

The scores sum to approximately one across the supplied candidates. This normalization makes them dependent on the candidate set. As a simple mathematical illustration, entailment logits 2 and 1 give scores approximately 0.731 and 0.269. Adding a third candidate with logit 2 changes the first score to approximately 0.422. These values illustrate the formula; they are not measured outputs from the project.

The application selects the top-ranked label, but a normalized score is not a calibrated guarantee of correctness. A system with only unsuitable categories will still produce a ranking. A rejection policy requires separate design and evaluation; simply treating a high score as certainty would overlook how the candidate set shapes that score.

4.3 Algorithm

Step 1: Validate and clean the passage and candidate labels.

Step 2: Load or reuse the NLI pipeline and obtain its tokenizer.

Step 3: Construct every passage-hypothesis pair and check its encoded length.

Step 4: Run the pipeline with the fixed template and selected scoring mode.

Step 5: Return ranked labels and aligned scores with the model configuration.

Step 6: Display or save the result; record execution failures separately.

4.4 Evaluation procedure

A larger evaluation should use a separately assembled labelled collection representing the intended news domain. One feasible college experiment is 40 passages, with 10 per topic, selected independently of the demonstration sentences. The category definitions and template must be fixed before final testing. If examples are used to choose a template or rejection threshold, they belong to a development set and should not also serve as the final test set.

Accuracy measures the proportion of correct first-ranked predictions. Precision asks how many predicted members of a class are correct; recall asks how many true members were recovered. F1 combines precision and recall, and macro-F1 gives each class equal weight. A confusion matrix helps identify recurring category confusions. A majority-class baseline and, where practical, a trained small classifier provide useful comparison points.

$$Accuracy = correct\ predictions / total\ test\ examples \tag{3}$$

$$F1\ for\ a\ class = 2\ TP / (2\ TP + FP + FN) \tag{4}$$

The evaluation should also examine negation, paraphrases, mixed topics, unknown entities and passages whose correct category is absent. Report the label policy, checkpoint, template, library versions, hardware and timing procedure. A perfect result on a small easy set establishes that those examples worked, not that the model understands all news. Test-set overlap with historical pretraining may be unknown and should not be ruled out without evidence.

4.5 Independent multiple-topic scoring

With multi_label=True, the pipeline compares entailment and contradiction separately for each candidate [5]. If e_i and d_i are their logits, the independent score is:

$$s_i = exp(e_i) / [exp(e_i) + exp(d_i)] \quad (5)$$

The scores no longer need to sum to one across topics. This setting can represent a passage relevant to several descriptions, but it does not itself establish which threshold defines a selected topic. Our interface displays the ranked relevance scores. Automatic selection and its precision-recall trade-off would need a documented policy evaluated on appropriate multiple-label references.

4.6 Robustness and reproducibility checks

Useful comparisons change one factor at a time. Hold the passage fixed while replacing a brief label with a more explicit description. Hold labels fixed while paraphrasing the passage. Add an irrelevant candidate and inspect score changes. These experiments reveal sensitivity to formulation, but should be reported as exploratory checks unless they form a fixed evaluation protocol.

Long documents require a deliberate policy. Splitting into passages preserves more content than simply discarding a suffix, but averaging passage scores may dilute a short decisive section. Maximum scores can overemphasize isolated sentences. These aggregation rules should be assessed against document-level references; none is implemented or empirically validated in the current project.

5. Implementation

5.1 Software environment

The project uses Python with PyTorch for model execution, Transformers for the task pipeline, Streamlit for the browser interface and Pandas for result tables. The recorded local environment used Transformers 4.57.6, Streamlit 1.65.0, Pandas 3.0.6 and CPU-only PyTorch 2.14.1. These are the versions used for the project, not a claim about the newest available releases.

The repository separates application requirements from the CPU PyTorch requirements. Its README explains virtual-environment setup and activation. Model weights are downloaded from Hugging Face when needed and are not stored in the GitHub repository. The first run therefore requires connectivity; cached model files can support subsequent local use.

5.2 Core implementation and code explanation

Listing 1 calls the actual project backend from inside the repository folder. It is deliberately short so that the application logic remains easy to identify.

Listing 1. Minimal use of the implemented NewsClassifier backend.

```
from classifier import NewsClassifier

model = NewsClassifier(device=-1)
text = "The team won the final cricket match."
labels = ["sports", "technology", "politics", "education"]

result = model.classify(text, labels, multi_label=False)
for label, score in zip(result["labels"], result["scores"]):
    print(f"{label:12s} {score:.4f}")
```

NewsClassifier stores the execution device, optional checkpoint revision and a pipeline initially set to None. Its load method constructs a zero-shot-classification pipeline using facebook/bart-large-mnli only when required. The classify method validates inputs, checks each tokenized passage-hypothesis pair and then performs inference. There is no fitting stage: calling classify uses the existing model weights.

The validation function uses strip to remove surrounding whitespace and casefold to compare labels without capitalization differences. A set detects duplicates. ValueError represents an invalid request. The return dictionary converts scores to ordinary floating-point values for JSON serialization and keeps model metadata with the output. Formatting scores to four decimal places affects presentation, not the internal prediction.

5.3 Repository structure and execution

Table 4. Main files in the public implementation.

File	Responsibility
classifier.py	Input validation, model loading and classification
app.py	Browser form, score table, chart and download
cli.py	Terminal arguments, JSON output and error reporting
evaluate.py	Fixed twelve-example demonstration evaluation
examples/	Expected categories and recorded real predictions
tests/	Backend and interface behaviour checks

From an activated project environment, streamlit run app.py starts the interface. The command python cli.py "The team won the final cricket match." performs a terminal prediction. The --labels option supplies categories, --multi-label changes the scoring mode, --output saves JSON and --revision can specify a checkpoint commit. The default --device value -1 selects CPU execution.

5.4 Interface and operational considerations

The interface collects comma-separated labels and converts them into a list. A form waits for submission, while a cached classifier object avoids repeated initialization. Session state retains the latest successful result. The page displays a table and score chart; a download button exports the dictionary as JSON. Multiple-topic mode displays relevance rather than automatically assigning a set of categories.

The interface clears a stale result after an invalid request and shows the error. The command line prints errors separately and returns a failure exit code. These behaviours prevent a failed request from being presented as a new prediction. Download, memory and runtime failures are operational issues; they should not be added to the confusion matrix as though they were valid category decisions.

5.5 Verification and interpretation of tests

The seven local tests cover backend validation, the expected inference contract and interface behaviours such as result display and invalid submission. Controlled replacements for the backend make these tests quick and deterministic. They do not measure whether BART's semantic predictions are correct. GitHub Actions runs the dependency-free checks; interface checks require Streamlit.

Separately, evaluate.py performs real inference on the twelve fixed examples and writes examples/sample_results.json. This separation is essential: software verification checks whether the application handles requests properly, while model evaluation checks the predictions produced on a defined collection. Neither substitutes for the other.

6. Result

6.1 Qualitative case analysis

Real CPU inference was completed on October 12, 2026. The checkpoint revision recorded in the result file is d7645e127eaf1aefc7862fd59a17a5aa8558b8ce. The twelve hand-authored demonstration passages contain three expected examples for each of the four topics. The label set and hypothesis template are stored with their predictions.

The passage "The cricket team won the championship after a close final match." was classified as sports. Its recorded category scores are shown below. Unlike a reference-category illustration, these values come from the real saved pipeline output.

Table 5. Measured prediction for the saved cricket-championship example.

Ranked topic	Recorded score, rounded
sports	0.9779
technology	0.0131
education	0.0053
politics	0.0037

The high sports score is consistent with this straightforward example. It is a relative preference among the four supplied topics, not evidence of 97.79% accuracy on arbitrary passages. Additional mixed-topic and irrelevant-label cases are required before drawing conclusions about difficult inputs.

6.2 Quantitative demonstration results

Table 6. Results of the small implementation demonstration.

Measure	Observed value
Demonstration examples	12
Correct first-ranked predictions	12
Accuracy on this set	100%
Macro-F1 on this set	1.00

All twelve predictions matched their expected categories. This establishes successful execution on the chosen sentences. The examples are short and hand-authored, so the result is not an independent news benchmark and should not be generalized. No larger evaluation, baseline comparison or statistical reliability estimate is claimed here. The 40-passage procedure in Section 4 is proposed future evaluation.

6.3 Interpretation and limitations

Several factors can alter the result: the category wording, the template, the domain, the candidate count and the passage's ambiguity. A funding announcement for sports education may legitimately concern sports, politics and education. A single reference category depends on the annotation policy. Without that policy, apparent model errors may partly reflect inconsistent human labels.

NLI-based scores also do not establish factual truth. A fabricated article can still be about politics. The model classifies what a passage expresses; it does not independently verify the event. Missing categories, biased linguistic associations and unfamiliar terminology are further sources of unreliable decisions. Human review and explicit uncertainty handling may be appropriate where mistakes have significant consequences.

6.4 Conclusion

The implementation demonstrates how a pretrained NLI checkpoint can categorize text using labels supplied at prediction time. The educational contribution is the complete workflow: task definition, meaningful label representation, validation, inference, result recording and interpretation. Its flexibility comes from textual hypotheses rather than from training a new output head for each topic set. Broader usefulness remains an empirical question requiring independent and more varied evaluation.

7. Current Status and Future Directions

7.1 Position of NLI-based classification today

The literature and tools in this section were reviewed on October 13, 2026. NLI-based classification remains a clear baseline for studying semantic label comparison. BART-MNLI is an established checkpoint, but it should not be described as the newest model or the universal best approach. Current comparisons include embedding systems, rerankers and instruction-tuned language models alongside NLI cross-encoders [7].

The practical question has shifted from whether zero-shot categorization is possible to which approach offers an acceptable combination of quality, computation, flexibility and reproducibility for a particular task. A small local system may favour a different checkpoint from a high-throughput document service. This report reviews representative evidence rather than claiming an exhaustive inventory of every available model.

7.2 Recent comparative evidence

Edwards and Camacho-Collados compared prompting and fine-tuning approaches across 16 classification datasets in 2024 [6]. Their results show that smaller fine-tuned models can outperform prompted larger models in evaluated settings. The implication is that avoiding target-task training is a useful constraint, not an automatic performance advantage.

The BTZSC study published at ICLR 2026 compares four model families across 22 datasets and 38 checkpoints [7]. It reports strong results for modern rerankers and competitive quality-latency trade-offs for embedding models. These findings broaden the comparison beyond NLI. They are results from that study's benchmark and cannot be directly compared with our 12/12 demonstration, which uses different data and a much smaller task.

7.3 Instruction prompting and structured outputs

An instruction-following model can receive a passage, category definitions and a request to return one category. This changes the interface from scoring explicit pairs to producing text. The application must then ensure that the response is one of the allowed labels and handle unsupported or malformed output. Generated explanations may help inspection, but a fluent explanation is not independent proof of a correct decision.

Few-shot examples can improve task specification, but introduce demonstrations and change the learning setting. Prompt tuning on labelled development cases should also be disclosed. For a fair comparison, use the same final test passages, category policy and reporting rules across methods. Compare latency and resources as well as accuracy, and state whether execution is local or depends on a hosted service.

7.4 Open challenges

Important challenges include category overlap, calibration, domain change, multilingual transfer and very large label sets. Calibration concerns whether a numerical confidence value corresponds to observed correctness. A rejection rule may use a score or a gap between the top candidates, but it needs held-out evidence. A threshold chosen after inspecting final test errors would give an overly favourable result.

Reproducibility also requires identifying changing dependencies and data. Model revisions, templates, label descriptions and evaluation examples should be retained with results. Sensitive passages and biased category systems raise questions beyond model accuracy: which labels should exist, who reviews mistakes and how corrections are recorded. These are application design issues that a successful pipeline call cannot resolve by itself.

7.5 Future work and learning outcomes

A feasible next stage is to execute the independent evaluation proposed in Section 4, add mixed-topic references and compare a second NLI checkpoint with an embedding baseline. Later work can assess long-document aggregation and evaluated rejection rules. Such extensions should each have a measurable goal rather than being added only to increase the feature count.

After studying this topic, a student should be able to distinguish zero-shot use from pretraining, explain a passage-hypothesis comparison, interpret the two scoring modes and describe why a small demonstration is not a general accuracy estimate. The field combines language representation, task formulation, software design and evaluation; understanding all four is more informative than memorizing a model name.

8. Reference

[1] W. Yin, J. Hay and D. Roth, "Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach," EMNLP-IJCNLP, pp. 3914-3923, 2019. https://aclanthology.org/D19-1404/.

[2] M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," ACL, pp. 7871-7880, 2020. https://aclanthology.org/2020.acl-main.703/.

[3] A. Williams, N. Nangia and S. R. Bowman, "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference," NAACL-HLT, pp. 1112-1122, 2018. https://aclanthology.org/N18-1101/.

[4] Hugging Face, "facebook/bart-large-mnli," model card. Accessed October 13, 2026. https://huggingface.co/facebook/bart-large-mnli.

[5] Hugging Face, "Pipelines: ZeroShotClassificationPipeline," Transformers 4.57.1 documentation. Accessed October 13, 2026. https://huggingface.co/docs/transformers/v4.57.1/en/main_classes/pipelines#transformers.ZeroShotClassificationPipeline.

[6] A. Edwards and J. Camacho-Collados, "Language Models for Text Classification: Is In-Context Learning Enough?," LREC-COLING, pp. 10058-10072, 2024. https://aclanthology.org/2024.lrec-main.879/.

[7] I. Aarab, "BTZSC: A Benchmark for Zero-Shot Text Classification Across Cross-Encoders, Embedding Models, and Rerankers," ICLR, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/417e1c15b3d49852fceded8aa104107d-Abstract-Conference.html.

Project implementation

Source code, setup instructions and recorded demonstration results:
https://github.com/Katori-bo/zero-shot-news-classifier